

Project Executable Files

Date	06 July 2026
Team ID	xxxxxx
Project Name	Voice Based Concept Understanding Analyser
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	Yes
5	Environment configuration file (.env.example similar)	Yes
6	Deployed application URL (if hosted)	No
7	APK / Executable binary (if applicable for mobile/desktop apps)	No
8	Dockerfile / Containerization config (if applicable)	No
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	No

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

```

vbcua/
├── app.py                # Main Streamlit application (Home → Upload → Result → History
pages)
├── config.py             # Central configuration (model settings, scoring weights, thresholds)
├── requirements.txt      # Python dependencies
├── README.md            # Setup and usage documentation
├── .streamlit/
│   └── config.toml      # UI theme configuration
├── data/
│   ├── reference_concepts.json # Reference explanations + key points for each concept
│   └── sessions.db       # SQLite database (stores evaluation history)
├── modules/             # Core Intelligence Layer
│   ├── speech_to_text.py  # Whisper-based speech-to-text transcription
│   ├── semantic_understanding.py # Sentence-BERT semantic similarity + key-point coverage
│   ├── filler_analysis.py  # NLTK-based filler word detection
│   ├── audio_features.py  # Librosa-based pause detection, RMS energy, waveform plotting
│   ├── scoring.py         # Combines all metrics into Similarity/Speech/Final scores
│   └── report_generator.py # ReportLab PDF report generation
├── utils/               # Supporting utilities
│   ├── storage.py        # SQLite database operations (save/retrieve sessions)
│   └── helpers.py        # File handling, audio validation (SoundFile)
├── assets/              # Auto-generated waveform images
└── tests/               # Unit tests
    ├── conftest.py       # Test configuration
    ├── test_filler_analysis.py # Tests for filler word detection
    └── test_scoring.py   # Tests for scoring logic
  
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	Not yet deployed — currently runs locally via Streamlit. [Add a URL here if you deploy to Streamlit Community Cloud, Render, etc.]
Login Credentials (Demo)	Not applicable — no authentication required; the app is open-access for the evaluator
Platform / Hosting Provider	Local (Streamlit) — <i>[Update if you deploy, e.g. "Streamlit Community Cloud"]</i>

Repository Link	https://github.com/Harshithamakina/SmartBridge-Voice_Based_Concept_Understanding_Analyser
Demo Video Link	NO

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

- ❑ Clone or download the project folder and navigate into it:
cd vbcua
- ❑ (Recommended) Create and activate a virtual environment:
python -m venv venv
venv\Scripts\activate (Windows)
- ❑ Install dependencies:
pip install -r requirements.txt
- ❑ Install FFmpeg separately (required by Whisper for audio decoding) and ensure it's added to your system PATH.
- ❑ Run the application:
streamlit run app.py
- ❑ The app opens at <http://localhost:8501> in your browser. First run downloads the Whisper and Sentence-BERT models (~220MB total) — requires internet access once.
- ❑ Select a concept, upload or record a spoken explanation, click **Analyze**, view the results, and download the PDF report.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Requires FFmpeg installed separately for decoding compressed audio formats (mp3/m4a)	Documented in README; not bundled with the app due to OS-specific installers
2	Whisper's "base" model prioritizes speed over transcription accuracy on CPU-only machines	Can switch to "small" or "medium" model in <code>config.py</code> if more compute is available
3	Limited to a fixed set of 5 pre-defined reference concepts	Easily extendable — new concepts can be added directly to <code>data/reference_concepts.json</code> without code changes